

Model Architecture

Setup

```
from google.colab import drive
drive.mount('/content/drive')

!git clone https://github.com/prajwl7676/mini-clip-DLAI.git

%cd /content/mini-clip-DLAI

!pip install -q -r requirements.txt
```

```
Mounted at /content/drive
Cloning into 'mini-clip-DLAI'...
remote: Enumerating objects: 13, done.
remote: Counting objects: 100% (13/13), done.
remote: Compressing objects: 100% (11/11), done.
remote: Total 13 (delta 1), reused 12 (delta 0), pack-reused 0 (from 0)
Receiving objects: 100% (13/13), 2.31 MiB | 7.27 MiB/s, done.
Resolving deltas: 100% (1/1), done.
/content/mini-clip-DLAI
===== 44.4/44.4 kB 4.3 MB/s eta 0:00:00
===== 61.0/61.0 kB 6.4 MB/s eta 0:00:00
===== 57.6/57.6 kB 5.9 MB/s eta 0:00:00
===== 906.4/906.4 MB 1.8 MB/s eta 0:00:00
===== 7.2/7.2 MB 117.5 MB/s eta 0:00:00
===== 9.7/9.7 MB 107.2 MB/s eta 0:00:00
===== 480.6/480.6 kB 44.3 MB/s eta 0:00:00
===== 4.5/4.5 MB 138.0 MB/s eta 0:00:00
===== 18.0/18.0 MB 103.5 MB/s eta 0:00:00
===== 8.3/8.3 MB 98.3 MB/s eta 0:00:00
===== 78.3/78.3 kB 6.6 MB/s eta 0:00:00
===== 12.9/12.9 MB 101.2 MB/s eta 0:00:00
===== 363.4/363.4 MB 6.2 MB/s eta 0:00:00
===== 13.8/13.8 MB 113.5 MB/s eta 0:00:00
===== 24.6/24.6 MB 84.1 MB/s eta 0:00:00
===== 883.7/883.7 kB 59.5 MB/s eta 0:00:00
===== 664.8/664.8 MB 1.3 MB/s eta 0:00:00
===== 211.5/211.5 MB 3.3 MB/s eta 0:00:00
===== 56.3/56.3 MB 13.6 MB/s eta 0:00:00
===== 127.9/127.9 MB 9.3 MB/s eta 0:00:00
===== 207.5/207.5 MB 6.3 MB/s eta 0:00:00
===== 188.7/188.7 MB 6.7 MB/s eta 0:00:00
===== 21.1/21.1 MB 44.1 MB/s eta 0:00:00
===== 99.1/99.1 kB 12.0 MB/s eta 0:00:00
===== 6.2/6.2 MB 63.3 MB/s eta 0:00:00
===== 209.6/209.6 MB 6.3 MB/s eta 0:00:00
===== 179.3/179.3 kB 19.5 MB/s eta 0:00:00
===== 566.4/566.4 kB 43.7 MB/s eta 0:00:00
===== 3.1/3.1 MB 96.9 MB/s eta 0:00:00
```

```
ERROR: pip's dependency resolver does not currently take into account all the packages that are installed. This behaviour is t
hdbscan 0.8.43 requires scikit-learn>=1.6, but you have scikit-learn 1.5.2 which is incompatible.
jaxlib 0.7.2 requires numpy>=2.0, but you have numpy 1.26.4 which is incompatible.
gcsfs 2025.3.0 requires fsspec==2025.3.0, but you have fsspec 2024.9.0 which is incompatible.
cupy-cuda12x 14.0.1 requires numpy<2.6,>=2.0, but you have numpy 1.26.4 which is incompatible.
opencv-contrib-python 4.13.0.92 requires numpy>=2; python_version >= "3.9", but you have numpy 1.26.4 which is incompatible.
jax 0.7.2 requires numpy>=2.0, but you have numpy 1.26.4 which is incompatible.
rasterio 1.5.0 requires numpy>=2, but you have numpy 1.26.4 which is incompatible.
opencv-python-headless 4.13.0.92 requires numpy>=2; python_version >= "3.9", but you have numpy 1.26.4 which is incompatible.
shap 0.51.0 requires numpy>=2, but you have numpy 1.26.4 which is incompatible.
xarray-einstats 0.10.0 requires numpy>=2.0, but you have numpy 1.26.4 which is incompatible.
dataproc-spark-connect 1.1.0 requires tqdm>=4.67, but you have tqdm 4.66.6 which is incompatible.
opencv-python 4.13.0.92 requires numpy>=2; python_version >= "3.9", but you have numpy 1.26.4 which is incompatible.
pytensort 2.38.3 requires numpy>=2.0, but you have numpy 1.26.4 which is incompatible.
tobler 0.14.0 requires numpy>=2.0, but you have numpy 1.26.4 which is incompatible.
tobler 0.14.0 requires tqdm>=4.67, but you have tqdm 4.66.6 which is incompatible.
tiff file 2026.4.11 requires numpy>=2.0, but you have numpy 1.26.4 which is incompatible.
umap-learn 0.5.12 requires scikit-learn>=1.6, but you have scikit-learn 1.5.2 which is incompatible.
```

```
import sys
sys.path.insert(0, '/content/mini-clip-DLAI')
```

```

import torch
import torch.nn as nn
import torch.nn.functional as F
import torchvision.models as tv_models
import matplotlib.pyplot as plt
import matplotlib
import numpy as np
from transformers import DistilBertModel, DistilBertTokenizer
from datasets import load_dataset

device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
print(f'Device : {device}')
if device.type == 'cuda':
    print(f'GPU      : {torch.cuda.get_device_name(0)}')

```

```

Device : cuda
GPU      : Tesla T4

```

✓ Image Encoder with projection head

```

EMBEDDING_DIM = 256 # shared dimensionality for both encoders

class ImageEncoder(nn.Module):
    def __init__(self, embedding_dim=EMBEDDING_DIM, freeze_layers=True):
        super().__init__()

        resnet = tv_models.resnet50(weights=tv_models.ResNet50_Weights.IMAGENET1K_V1)
        self.backbone = nn.Sequential(*list(resnet.children())[:-1])
        # backbone output: (B, 2048, 1, 1)

        self.projection = nn.Sequential(
            nn.Linear(2048, embedding_dim, bias=False),
            nn.LayerNorm(embedding_dim),
        )

        if freeze_layers:
            for layer in [resnet.conv1, resnet.bn1, resnet.layer1,
                          resnet.layer2, resnet.layer3]:
                for param in layer.parameters():
                    param.requires_grad = False

        def forward(self, images):
            features = self.backbone(images).flatten(start_dim=1) # (B, 2048)
            embeddings = self.projection(features)                # (B, embedding_dim)
            return F.normalize(embeddings, p=2, dim=-1)            # L2-normalised

image_encoder = ImageEncoder().to(device)
print('ImageEncoder created.')

```

Downloading: "<https://download.pytorch.org/models/resnet50-0676ba61.pth>" to /root/.cache/torch/hub/checkpoints/resnet50-0676ba61.pth [100%|██████████| 97.8M/97.8M [00:00<00:00, 170MB/s]]
ImageEncoder created.

✓ Test the Image Encoder with a fake batch

```

BATCH_SIZE = 8

fake_images = torch.randn(BATCH_SIZE, 3, 224, 224).to(device)

with torch.no_grad():
    image_embeddings = image_encoder(fake_images)

print(f'Input shape : {fake_images.shape}') # (8, 3, 224, 224)
print(f'Output shape : {image_embeddings.shape}') # (8, 256)
print(f'L2 norms (should all be ~1.0): {image_embeddings.norm(dim=-1).tolist()}')

assert image_embeddings.shape == (BATCH_SIZE, EMBEDDING_DIM), 'Wrong output shape!'

```

```
assert torch.allclose(image_embeddings.norm(dim=-1), torch.ones(BATCH_SIZE).to(device), atol=1e-5), 'Not L2-normalised!'
print('\nAll image encoder checks passed.')
```

```
Input shape : torch.Size([8, 3, 224, 224])
Output shape : torch.Size([8, 256])
L2 norms (should all be ~1.0): [1.0, 0.9999999403953552, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0]

All image encoder checks passed.
```

Image Encoder parameter breakdown

```
def count_params(model, name='Model'):
    total = sum(p.numel() for p in model.parameters())
    trainable = sum(p.numel() for p in model.parameters() if p.requires_grad)
    frozen = total - trainable
    print(f'{name}')
    print(f' Total params      : {total:>12,}')
    print(f' Trainable params : {trainable:>12,} ({100*trainable/total:.1f}%)')
    print(f' Frozen params    : {frozen:>12,} ({100*frozen/total:.1f}%)')
    return trainable

count_params(image_encoder, 'ImageEncoder')
```

```
ImageEncoder
Total params      : 24,032,832
Trainable params  : 15,489,536 (64.5%)
Frozen params     : 8,543,296 (35.5%)
15489536
```

Text Encoder with projection head

```
class TextEncoder(nn.Module):
    def __init__(self, embedding_dim=EMBEDDING_DIM, model_name='distilbert-base-uncased', freeze_layers=True):
        super().__init__()

        self.bert = DistilBertModel.from_pretrained(model_name)

        if freeze_layers:
            # 1. Freeze early transformer layers (0 and 1 out of 6)
            for layer in self.bert.transformer.layer[:2]:
                for param in layer.parameters():
                    param.requires_grad = False

            # 2. Also freeze the heavy text embeddings lookup table
            for param in self.bert.embeddings.parameters():
                param.requires_grad = False

        # DistilBERT hidden size = 768
        self.projection = nn.Sequential(
            nn.Linear(768, embedding_dim, bias=False),
            nn.LayerNorm(embedding_dim),
        )

    def forward(self, input_ids, attention_mask):
        output = self.bert(input_ids=input_ids, attention_mask=attention_mask)
        cls_output = output.last_hidden_state[:, 0, :] # [CLS] token: (B, 768)
        embeddings = self.projection(cls_output)      # (B, embedding_dim)
        return F.normalize(embeddings, p=2, dim=-1)

text_encoder = TextEncoder().to(device)
print('TextEncoder created.')
```

TextEncoder created.

Test the Text Encoder with fake tokens

```
MAX_LENGTH = 77
```

```
# Vocabulary size of DistilBERT = 30,522
fake_input_ids = torch.randint(0, 30522, (BATCH_SIZE, MAX_LENGTH)).to(device)
fake_attention_mask = torch.ones(BATCH_SIZE, MAX_LENGTH, dtype=torch.long).to(device)

with torch.no_grad():
    text_embeddings = text_encoder(fake_input_ids, fake_attention_mask)

print(f'input_ids shape : {fake_input_ids.shape}') # (8, 77)
print(f'Output shape : {text_embeddings.shape}') # (8, 256)
print(f'L2 norms : {text_embeddings.norm(dim=-1).tolist()}')

assert text_embeddings.shape == (BATCH_SIZE, EMBEDDING_DIM)
print('\nAll text encoder checks passed.')

input_ids shape : torch.Size([8, 77])
Output shape : torch.Size([8, 256])
L2 norms : [1.0, 1.0, 1.0, 1.0, 0.9999999403953552, 1.0, 1.0, 1.0]

All text encoder checks passed.
```

CLIPModel

```
class CLIPModel(nn.Module):
    def __init__(self, embedding_dim=EMBEDDING_DIM, temperature_init=0.07):
        super().__init__()
        self.image_encoder = ImageEncoder(embedding_dim)
        self.text_encoder = TextEncoder(embedding_dim)
        # Store log( $\tau$ ) so  $\tau = \exp(\log \tau)$  is always positive
        self.log_temperature = nn.Parameter(torch.tensor(temperature_init).log())

    @property
    def temperature(self):
        return self.log_temperature.exp()

    def encode_image(self, images):
        return self.image_encoder(images)

    def encode_text(self, input_ids, attention_mask):
        return self.text_encoder(input_ids, attention_mask)

    def forward(self, images, input_ids, attention_mask):
        img_emb = self.image_encoder(images)
        txt_emb = self.text_encoder(input_ids, attention_mask)
        return img_emb, txt_emb, self.temperature

model = CLIPModel().to(device)
print('CLIPModel created.')
print(f'Initial temperature  $\tau = \{{\text{model.temperature.item():.4f}}\}$ ')

CLIPModel created.
Initial temperature  $\tau = 0.0700$ 
```

Full end-to-end shape test

```
with torch.no_grad():
    img_emb, txt_emb, tau = model(fake_images, fake_input_ids, fake_attention_mask)

print('=== CLIPModel forward pass ===')
print(f'Image embeddings : {img_emb.shape}') # (8, 256)
print(f'Text embeddings : {txt_emb.shape}') # (8, 256)
print(f'Temperature  $\tau$  : {tau.item():.4f}')

sim_matrix = img_emb @ txt_emb.T
print(f'Similarity matrix: {sim_matrix.shape}') # (8, 8)

# Sanity: diagonal should be the matched pairs
print(f'Diagonal values (matched pairs) : {sim_matrix.diag().tolist()}')
```

```

assert img_emb.shape == (BATCH_SIZE, EMBEDDING_DIM)
assert txt_emb.shape == (BATCH_SIZE, EMBEDDING_DIM)
assert sim_matrix.shape == (BATCH_SIZE, BATCH_SIZE)
print('\nAll CLIPModel checks passed.')

=== CLIPModel forward pass ===
Image embeddings : torch.Size([8, 256])
Text embeddings : torch.Size([8, 256])
Temperature  $\tau$  : 0.0700
Similarity matrix: torch.Size([8, 8])
Diagonal values (matched pairs) : [0.008224545046687126, -0.001114854123443365, -0.0009587886743247509, 0.015249417163431644,
All CLIPModel checks passed.

```

✓ Full parameter count

```

print('=' * 50)
img_t = count_params(model.image_encoder, 'ImageEncoder')
print()
txt_t = count_params(model.text_encoder, 'TextEncoder')
print()
total_t = sum(p.numel() for p in model.parameters() if p.requires_grad)
total_a = sum(p.numel() for p in model.parameters())
print('=' * 50)
print(f'CLIPModel total')
print(f' All params      : {total_a:>12,}')
print(f' Trainable params : {total_t:>12,} ({100*total_t/total_a:.1f}%)')
print(f' Approx GPU memory: ~{total_t * 4 / 1e6:.0f} MB (float32)')

```

```

=====
ImageEncoder
  Total params      : 24,032,832
  Trainable params  : 15,489,536 (64.5%)
  Frozen params     : 8,543,296 (35.5%)

TextEncoder
  Total params      : 66,560,000
  Trainable params  : 28,548,608 (42.9%)
  Frozen params     : 38,011,392 (57.1%)

=====
CLIPModel total
  All params      : 90,592,833
  Trainable params : 44,038,145 (48.6%)
  Approx GPU memory: ~176 MB (float32)

```

✓ Visualise similarity matrix on real data

```

from src.dataset import FlickrDataset, get_transform, build_loaders
from torch.utils.data import DataLoader

dataset_hf = load_dataset('nlphuji/flickr30k', split='test[:200]')
tokenizer = DistilBertTokenizer.from_pretrained('distilbert-base-uncased')

viz_ds = FlickrDataset(dataset_hf, tokenizer, get_transform(train=False))
viz_loader = DataLoader(viz_ds, batch_size=8, shuffle=False)

images_batch, tokens_batch = next(iter(viz_loader))
images_batch = images_batch.to(device)
input_ids = tokens_batch['input_ids'].to(device)
attn_mask = tokens_batch['attention_mask'].to(device)

print(f'Real batch loaded: {images_batch.shape}')

```

```

README.md: 100%                                641/641 [00:00<00:00, 64.7kB/s]

flickr30k.py:      2.51k/? [00:00<00:00, 211kB/s]

TEST/test/0000.parquet: 100%                    506M/506M [00:03<00:00, 152MB/s]
TEST/test/0001.parquet: 100%                    502M/502M [00:04<00:00, 135MB/s]
TEST/test/0002.parquet: 100%                    506M/506M [00:03<00:00, 164MB/s]
TEST/test/0003.parquet: 100%                    512M/512M [00:02<00:00, 213MB/s]
TEST/test/0004.parquet: 100%                    504M/504M [00:05<00:00, 89.4MB/s]
TEST/test/0005.parquet: 100%                    495M/495M [00:08<00:00, 65.3MB/s]
TEST/test/0006.parquet: 100%                    495M/495M [00:05<00:00, 113MB/s]
TEST/test/0007.parquet: 100%                    497M/497M [00:08<00:00, 63.0MB/s]
TEST/test/0008.parquet: 100%                    289M/289M [00:03<00:00, 65.7MB/s]

Generating test split: 100%                    31014/31014 [01:05<00:00, 491.90 examples/s]

tokenizer_config.json: 100%                    48.0/48.0 [00:00<00:00, 4.44kB/s]

vocab.txt:      232k/? [00:00<00:00, 6.29MB/s]

tokenizer.json:   466k/? [00:00<00:00, 8.47MB/s]

Real batch loaded: torch.Size([8, 3, 224, 224])

```

```

# Compute similarity matrix with untrained model
model.eval()
with torch.no_grad():
    img_emb, txt_emb, tau = model(images_batch, input_ids, attn_mask)
    sim = (img_emb @ txt_emb.T).cpu().numpy() # (8, 8)

# Get captions for labels
captions = [dataset_hf[i]['caption'][0][:30] + '...' for i in range(8)]

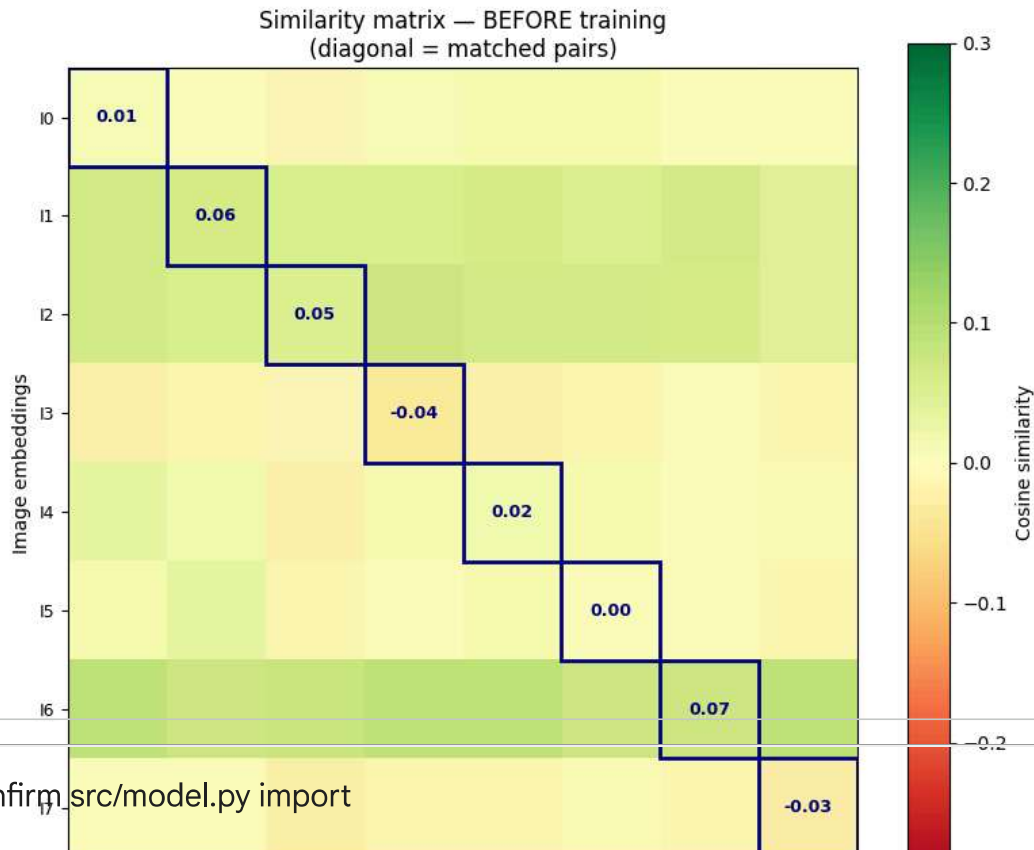
fig, ax = plt.subplots(figsize=(8, 7))
im = ax.imshow(sim, cmap='RdYlGn', vmin=-0.3, vmax=0.3)

ax.set_xticks(range(8))
ax.set_yticks(range(8))
ax.set_xticklabels([f'T{i}' for i in range(8)], rotation=45, ha='right', fontsize=9)
ax.set_yticklabels([f'I{i}' for i in range(8)], fontsize=9)
ax.set_xlabel('Text embeddings')
ax.set_ylabel('Image embeddings')
ax.set_title('Similarity matrix - BEFORE training\n(diagonal = matched pairs)')

# Highlight diagonal
for i in range(8):
    ax.add_patch(plt.Rectangle((i-0.5, i-0.5), 1, 1, fill=False,
                               edgecolor='navy', lw=2))
    ax.text(i, i, f'{sim[i,i]:.2f}', ha='center', va='center',
            fontsize=9, fontweight='bold', color='navy')

plt.colorbar(im, ax=ax, label='Cosine similarity')
plt.tight_layout()
plt.savefig('similarity_matrix_before_training.png', dpi=120, bbox_inches='tight')
plt.show()
print('Saved similarity_matrix_before_training.png')
print()
print('NOTE: Before training, the diagonal has no clear pattern.')
print('After training, the diagonal should be the brightest row in each column.')

```



✓ Confirm `src/model.py` import

```
from src.model import CLIPModel, count_parameters

model_from_src = CLIPModel(embedding_dim=256).to(device)

# Verify forward pass
with torch.no_grad():
    ie, te, t = model_from_src(fake_images, fake_input_ids, fake_attention_mask)

assert ie.shape == (BATCH_SIZE, 256)
assert te.shape == (BATCH_SIZE, 256)

counts = count_parameters(model_from_src)
print('src.model import works correctly.')
print(f'Trainable params : {counts["trainable"]}, ({counts["trainable_pct"]}%')
```